Blockchain Experiment 1

Name: Parth Narkar
D20A / 53

AIM: Cryptography in Blockchain, Merkle Root Tree Hash
Google Colab Link:
https://colab.research.google.com/drive/1aNbujr_dheDzFeVVK4JlGouzLUartyXw?usp=sharing

Theory:

1. Cryptographic Hash Function:
A cryptographic hash function converts input data into a fixed-length hash value. SHA-256 is commonly used in blockchain to ensure data integrity and security. A small change in input produces a completely different hash.

2. Merkle Tree:
A Merkle Tree is a binary tree in which transaction data is hashed and the hashes are combined repeatedly until a single hash, called the **Merkle Root**, is obtained.

3. Cryptographic Puzzle and Golden Nonce:
A cryptographic puzzle requires finding a **nonce** that produces a hash satisfying a specified condition, such as starting with a certain number of zeros. The nonce that satisfies this condition is called the **Golden Nonce**.

4. Working of Merkle Tree:
Transactions are first hashed individually. Pairs of hashes are then combined and hashed again. This process continues until only one hash remains, which is the Merkle Root.

5. Benefits of Merkle Tree:

- Efficient verification of transactions
- Ensures data integrity
- Reduces the amount of data required for verification
- Helps detect changes in transactions quickly

6. Use Cases:

Merkle Trees are used in **Bitcoin and other blockchain systems** to efficiently verify transactions and maintain the integrity of blocks.

Implementation:

1. **Program #1: Python program that uses the hashlib library to create the hash of a given string:**

[5] ✓ 3s

```
import hashlib                                                  #error statement

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()                       #error statement
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)                                      #error statement
```

```
Enter a string: Parth
Hash: 4ec1a76282e4f89a809b90cea83ef509a5eda8d8d128819905a7459830a2ebe5
```

2. **Program #2: Program to generate the required target hash with an input string and nonce**

[6] ✓ 10s

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce                               # Error

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)                                    # Error
```

```
Enter a string: Parth
Enter the nonce: 35
Hash Code: 80d0042e052d13fa83b549906a5036bfa749bc409d64e0cafa34a9cf9820941a
```

3. **Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string**

```
[8]
✓ 0s
```

```python
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)                    #Error
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):                          #Error
            print("Hash:",hash_code )
            return nonce

        nonce += 1

# Get user input
input_string = "Parth"
num_zeros = 1

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)                             #Error
```

```
Hash: 0b3714db66c59b0c78458e9e26e514cdfd9c0ea5d13ad48350729b5918cf8781
Input String: Parth
Leading Zeros: 1
```

4. Program 4: Generating Merkle Tree for given set of Transactions

```
[9]
✓ 0s
import hashlib

def build_merkle_tree(transactions):
    current_level_hashes = list(transactions) # Work with a mutable local copy

    if not current_level_hashes:
        return None

    if len(current_level_hashes) == 1:
        print(f"Level 0 (Root) hash: {current_level_hashes[0]}") # Only one element, so it's the root.
        return current_level_hashes[0]

    level_num = 0
    print(f"Level {level_num} (Leaf) hashes: {current_level_hashes}") # Print initial hashes (leaf level)

    # Recursive construction of the Merkle Tree
    while len(current_level_hashes) > 1:
        level_num += 1
        # Handle odd number of hashes by duplicating the last one
        if len(current_level_hashes) % 2 != 0:
            current_level_hashes.append(current_level_hashes[-1])

        next_level_hashes = []
        for i in range(0, len(current_level_hashes), 2):
            # Combine two hashes and hash the result
            combined = current_level_hashes[i] + current_level_hashes[i+1]
            hash_combined = hashlib.sha256(combined.encode('utf-8')).hexdigest()
            next_level_hashes.append(hash_combined) # Corrected placeholder

        print(f"Level {level_num} hashes: {next_level_hashes}") # Print intermediate hashes
        current_level_hashes = next_level_hashes # Move to the next level of hashes
```

```
[9]
✓ 0s
            hash_combined = hashlib.sha256(combined.encode('utf-8')).hexdigest()
            next_level_hashes.append(hash_combined) # Corrected placeholder

        print(f"Level {level_num} hashes: {next_level_hashes}") # Print intermediate hashes
        current_level_hashes = next_level_hashes # Move to the next level of hashes

    return current_level_hashes[0] # The final remaining hash is the Merkle Root

# Example usage
initial_transactions = [
    'Alice -> Bob : $200',
    'Bob -> Dave : $500',
    'Dave -> Eve : $100',
    'Eve -> Alice : $300',
    'Roo -> Bob : $50'
]

# Hash each initial transaction to form the leaf nodes of the Merkle Tree
hashed_leaf_nodes = [hashlib.sha256(t.encode('utf-8')).hexdigest() for t in initial_transactions]
print("Original Transactions:", initial_transactions)
print("---")
print("Hashed Leaf Nodes before building Merkle Tree:", hashed_leaf_nodes)
print("---")

merkle_root = build_merkle_tree(hashed_leaf_nodes)
print("---")
print("Final Merkle Root:", merkle_root) # Corrected placeholder
```

```
Original Transactions: ['Alice -> Bob : $200', 'Bob -> Dave : $500', 'Dave -> Eve : $100', 'Eve -> Alice : $300', 'Roo -> Bob : $50']
---
Hashed Leaf Nodes before building Merkle Tree: ['f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760', '768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46',
---
Level 0 (Leaf) hashes: ['f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760', '768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46', '43dd5729f00e01eea73a8
Level 1 hashes: ['471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9', 'a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7', '314ca746b3dafe63a1e0b939ec304
Level 2 hashes: ['ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee', '6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45']
Level 3 hashes: ['96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0']
---
Final Merkle Root: 96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0
```

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- SHA-256 Hashing provides a secure and unique fingerprint for any input, ensuring data integrity.

- Nonce and Target Hashing allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- Proof-of-Work Puzzle Solving shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- Merkle Tree Construction efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.